

LLM 생성 함수 변종을 활용한 퍼징 커버리지 향상 기법

장세창, 이아청, 김문주

2026.02.02



문제

- 퍼징 대상 프로그램은 항상 퍼징에 적합한 프로그램인가?

문제

- 퍼징에 적합한 프로그램은 어떠한 프로그램인가?

무작위로 시도해도 우연히 통과할 가능성이 있는 프로그램



복잡한 조건이 없는 프로그램

문제

- 퍼징 대상 프로그램은 항상 퍼징에 적합한 프로그램인가?

```
1: uint16_t magic = read_u16(fd);  
2: if (magic != 0x524C)  
3:     return ERROR;  
4: parse_body(fd);
```

상수 매칭

문제

- 퍼징 대상 프로그램은 항상 퍼징에 적합한 프로그램인가?

```
1: uint16_t magic = read_u16(fd);  
2: if (magic != 0x524C)  
3:     return ERROR;  
4: parse_body(fd);
```

상수 매칭

```
1: cnt = 0;  
2: while (read_byte(fd) == 0xFF)  
3:     cnt++;  
4: if (cnt == 50)  
5:     target();
```

누적 반복

문제

- 퍼징 대상 프로그램은 항상 퍼징에 적합한 프로그램인가?

```
1: uint16_t magic = read_u16(fd);  
2: if (magic != 0x524C)  
3:   return ERROR;  
4: parse_body(fd);
```

상수 매칭

```
1: cnt = 0;  
2: while (read_byte(fd) == 0xFF)  
3:   cnt++;  
4: if (cnt == 50)  
5:   target();
```

누적 반복

```
1: if (a > 0)  
2:   if (b > 0)  
3:     if (c > 0)  
4:       if (d > 0)  
5:         target();
```

결합 조건

문제

- 퍼징 대상 프로그램은 항상 퍼징에 적합한 프로그램인가?

```
1: uint16_t magic = read_u16(fd);  
2: if (magic != 0x524C)  
3:     return ERROR;  
4: parse_body(fd);
```

상수 매칭

```
1: cnt = 0;  
2: while (read_byte(fd) == 0xFF)  
3:     cnt++;  
4: if (cnt == 50)  
5:     target();
```

누적 반복

```
1: if (a > 0)  
2:     if (b > 0)  
3:         if (c > 0)  
4:             if (d > 0)  
5:                 target();
```

결합 조건

퍼징 대상 프로그램은 항상 퍼징에 적합하지 않다!

접근 방법

- LLM을 활용하여 프로그램을 퍼징에 적합한 **변종 프로그램**으로 수정하자!

```
1: uint16_t magic = read_u16(fd);  
2: if (magic != 0x524C)  
3:     return ERROR;  
4: parse_body(fd);
```

상수 매칭

```
1: cnt = 0;  
2: while (read_byte(fd) == 0xFF)  
3:     cnt++;  
4: if (cnt == 50)  
5:     target();
```

누적 반복

```
1: if (a > 0)  
2:     if (b > 0)  
3:         if (c > 0)  
4:             if (d > 0)  
5:                 target();
```

결합 조건

접근 방법

- LLM을 활용하여 프로그램을 퍼징에 적합한 **변종 프로그램**으로 수정하자!

```
1: uint16_t magic = read_u16(fd);  
2: if (magic != 0x524C)  
3:   return ERROR;  
4: parse_body(fd);
```

상수 매칭



```
1: uint16_t magic = read_u16(fd);  
2: if (magic != 0x524C)  
3: return ERROR;  
4: parse_body(fd);
```

상수 매칭 제거

```
1: cnt = 0;  
2: while (read_byte(fd) == 0xFF)  
3:   cnt++;  
4: if (cnt == 50)  
5:   target();
```

누적 반복

```
1: if (a > 0)  
2:   if (b > 0)  
3:     if (c > 0)  
4:       if (d > 0)  
5:         target();
```

결합 조건

접근 방법

- LLM을 활용하여 프로그램을 퍼징에 적합한 변종 프로그램으로 수정하자!

```
1: uint16_t magic = read_u16(fd);  
2: if (magic != 0x524C)  
3:   return ERROR;  
4: parse_body(fd);
```

상수 매칭



```
1: uint16_t magic = read_u16(fd);  
2: if (magic != 0x524C)  
3: return ERROR;  
4: parse_body(fd);
```

상수 매칭 제거

```
1: cnt = 0;  
2: while (read_byte(fd) == 0xFF)  
3:   cnt++;  
4: if (cnt == 50)  
5:   target();
```

누적 반복



```
1: cnt = 0;  
2: while (read_byte(fd) == 0xFF)  
3:   cnt++;  
4: if (cnt == 25)  
5:   target();
```

누적 반복 조건 완화

```
1: if (a > 0)  
2:   if (b > 0)  
3:     if (c > 0)  
4:       if (d > 0)  
5:         target();
```

결합 조건

접근 방법

- LLM을 활용하여 프로그램을 퍼징에 적합한 **변종 프로그램**으로 수정하자!

```
1: uint16_t magic = read_u16(fd);  
2: if (magic != 0x524C)  
3:   return ERROR;  
4: parse_body(fd);
```

상수 매칭



```
1: uint16_t magic = read_u16(fd);  
2: if (magic != 0x524C)  
3: return ERROR;  
4: parse_body(fd);
```

상수 매칭 제거

```
1: cnt = 0;  
2: while (read_byte(fd) == 0xFF)  
3:   cnt++;  
4: if (cnt == 50)  
5:   target();
```

누적 반복



```
1: cnt = 0;  
2: while (read_byte(fd) == 0xFF)  
3:   cnt++;  
4: if (cnt == 25)  
5:   target();
```

누적 반복 조건 완화

```
1: if (a > 0)  
2:   if (b > 0)  
3:     if (c > 0)  
4:       if (d > 0)  
5:         target();
```

결합 조건

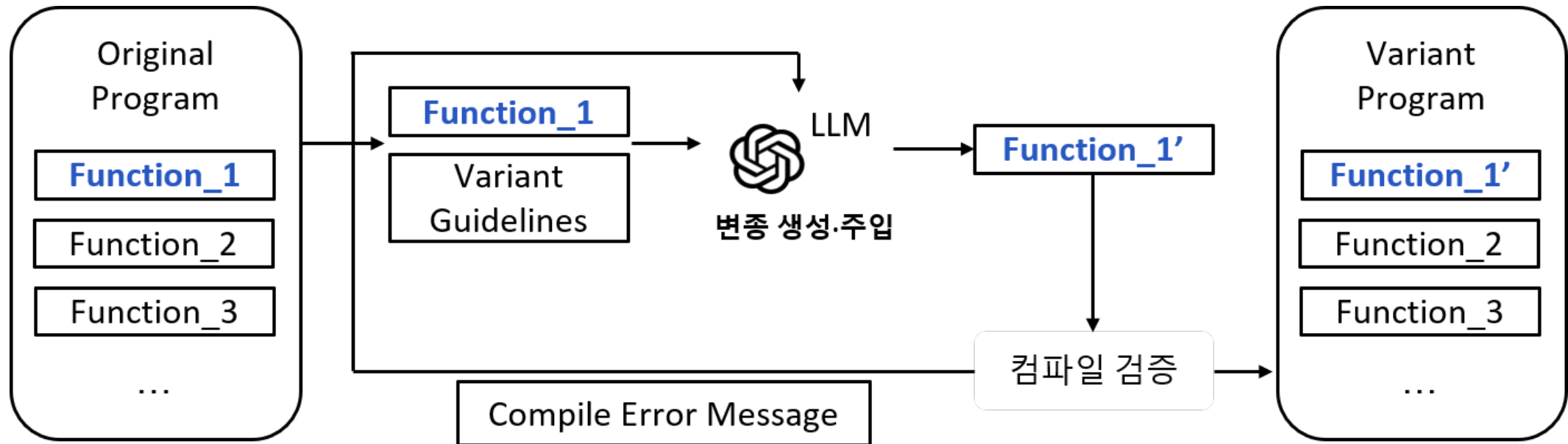


```
1: if (a > 0)  
2: if (b > 0)  
3: if (c > 0)  
4:   if (d > 0)  
5:     target();
```

결합 조건 축소

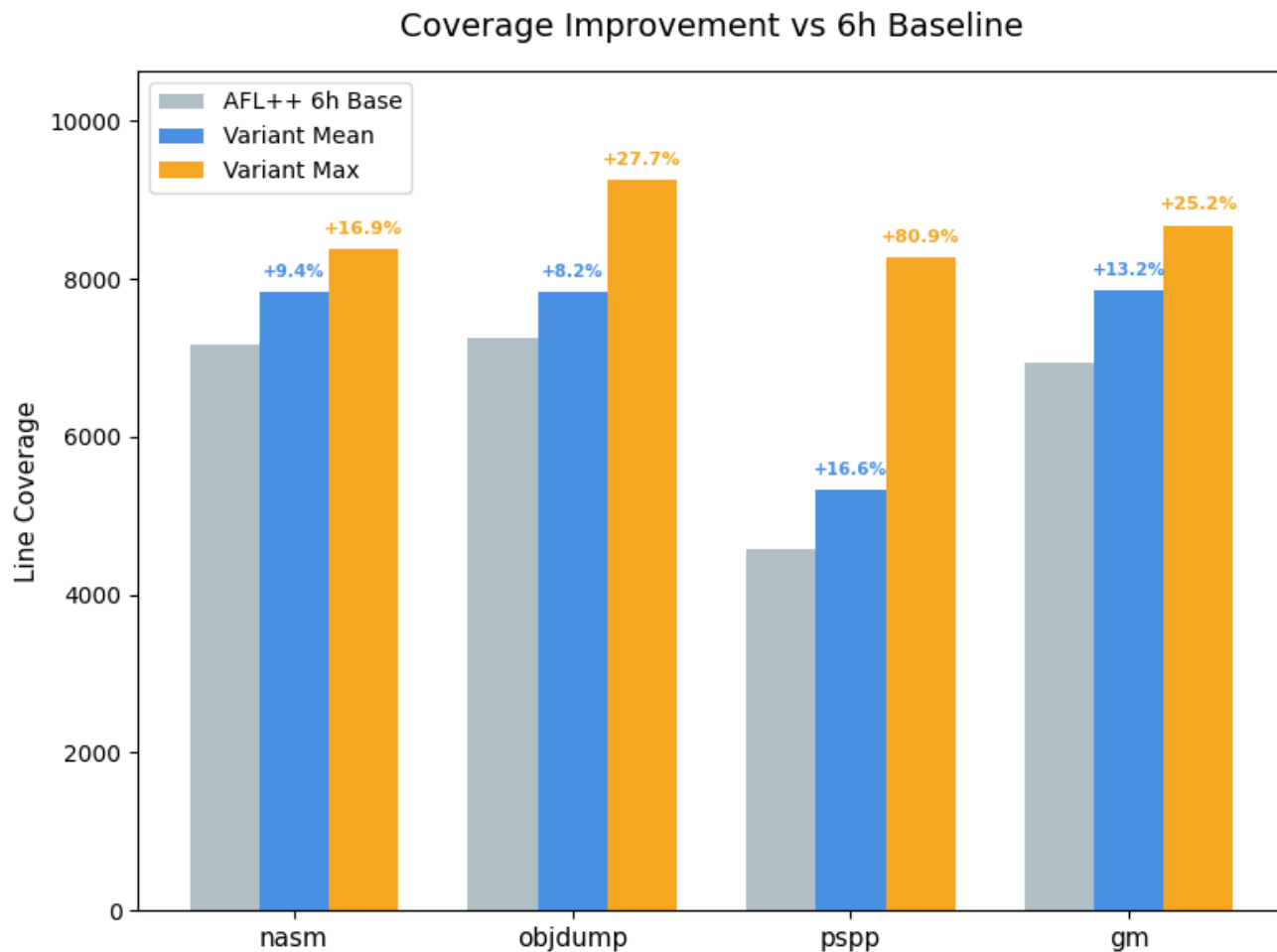
제안 시스템

- 변종을 통해 퍼저가 프로그램의 **숨겨진 행동**을 보도록 허용하는 침습적 접근
- LLM을 이용한 다양한 변종 생성을 통해 퍼징 커버리지 개선



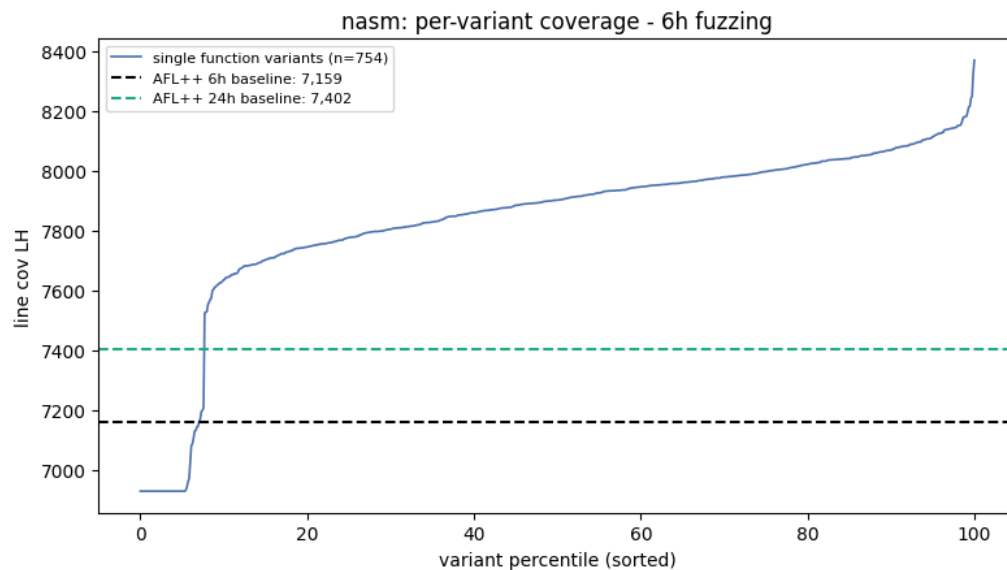
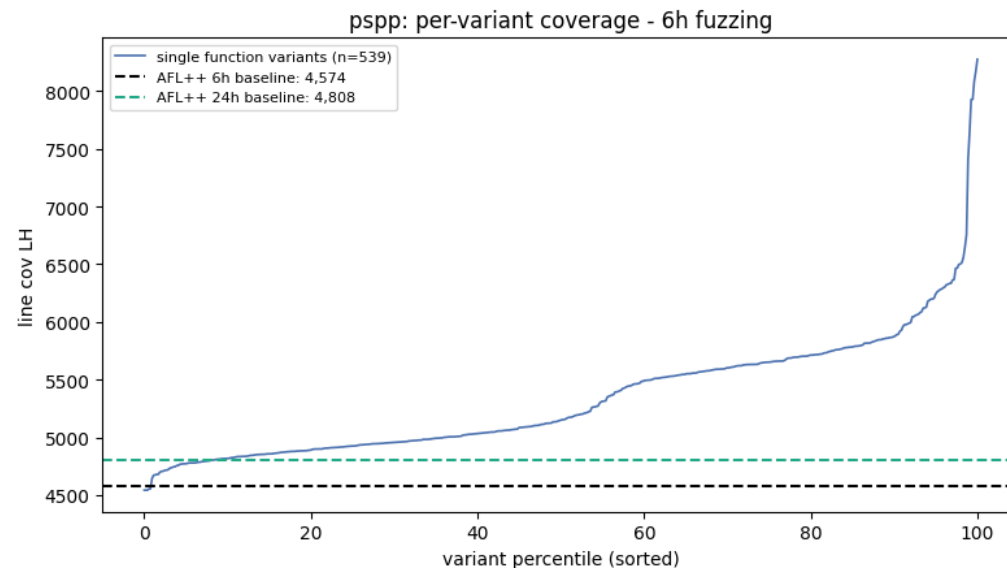
LLM 생성 변종은 퍼징 커버리지를 얼마나 개선하는가?

- 변종 프로그램 대상 6시간 퍼징 결과물을 원본 프로그램에 replay
- 4개의 subject에서 평균 **11.9%**의 커버리지 상승



실험 결과

- nasm과 pspp에서는 각각 **92%, 91%**의 생성된 변종이 **6시간 퍼징**만으로 **24시간 baseline 퍼징**을 앞선다.
- pspp의 경우 24시간 baseline 퍼징 대비 **72.1%** 커버리지 개선을 보이는 변종이 존재했다.



개선 방향

- 변종으로 인해 생성되는 다수의 crash 오탐
 - 변종 프로그램이 유발한 crash 29,991개 중 원본 재현 0개
- 최적 변종 예측 불가로 인한 스케줄링 부재
- 무한루프를 생성하거나 과도하게 프로그램의 흐름을 훼손하여 악영향을 미치는 변종 생성
